Python Object-Oriented Programming

Classes, Objects, Encapsulation, Inheritance, and Polymorphism

---

1. Introduction

Object-Oriented Programming (OOP) is a programming paradigm that organizes code around **objects** — bundles of data (attributes) and behaviour (methods) — rather than just functions and logic acting on separate data. Python is a fully object-oriented language: even simple values like integers and strings are objects. Understanding OOP allows you to model real-world entities (a Car, a BankAccount, a User) in code in a way that is organized, reusable, and easy to extend.

This module covers how to define classes and create objects from them, the special `__init__` constructor method, the role of `self`, instance attributes and methods, encapsulation, inheritance, method overriding, and polymorphism.

2. Learning Objectives

- Explain what object-oriented programming is and why it is useful.
- Define classes and instantiate objects from them.
- Use __init__ and self to initialize and work with instance data.
- Distinguish instance attributes from instance methods.
- Apply encapsulation to protect an object's internal state.
- Use inheritance to build specialized classes from a general base class.
- Override inherited methods and explain polymorphism with concrete examples.

3. Concept Explanations

3.1 Classes and Objects

A **class** is a blueprint that defines the attributes and behaviours an object of that type will have. An **object** (also called an instance) is a specific realization of that class, created by calling the class like a function. Multiple objects can be created from the same class, each with its own independent data.

```
class Dog:
    pass

buddy = Dog()      # buddy is an object (instance) of class Dog
max_dog = Dog()    # max_dog is a separate object of the same class
print(type(buddy))            # <class '__main__.Dog'>
print(buddy is max_dog)       # False, they are different objects
```

3.2 __init__ and self

`__init__` is a special method (a 'dunder', short for double underscore, method) automatically called when a new object is created. It is used to initialize the object's attributes. `self` refers to the specific instance being worked with, and must be the first parameter of every instance method, though Python passes it in automatically when you call the method — you never pass it explicitly.

```
class Dog:
    def __init__(self, name, breed):
        self.name = name      # instance attribute
        self.breed = breed    # instance attribute

buddy = Dog("Buddy", "Golden Retriever")
print(buddy.name)      # Buddy
print(buddy.breed)     # Golden Retriever
```

3.3 Instance Attributes and Instance Methods

Instance attributes are variables bound to a specific object (usually set in `__init__` via `self.attribute = value`). Instance methods are functions defined inside a class that operate on that object's data, accessed through `self`.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        return f"{self.name} says Woof!"

    def have_birthday(self):
        self.age += 1

buddy = Dog("Buddy", 3)
print(buddy.bark())         # Buddy says Woof!
buddy.have_birthday()
print(buddy.age)            # 4
```

3.4 Encapsulation

Encapsulation means bundling data and the methods that operate on it together, and restricting direct outside access to some of an object's internal details. Python uses a naming convention rather than strict enforcement: a single leading underscore (`_name`) signals 'internal use only', and a double leading underscore (`__name`) triggers name-mangling, making accidental external access harder.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance     # 'private' attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance
```

```
account = BankAccount("Alice", 100)
account.deposit(50)
print(account.get_balance())    # 150
# print(account.__balance)      # AttributeError (name-mangled)
```

3.5 Inheritance

Inheritance allows a new class (the **subclass** or **child class**) to reuse and extend the attributes and methods of an existing class (the **superclass** or **parent class**). This avoids duplicating code across related classes and models 'is-a' relationships, such as 'a Cat is an Animal'.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Cat(Animal):          # Cat inherits from Animal
    def purr(self):
        return f"{self.name} purrs"

whiskers = Cat("Whiskers")
print(whiskers.speak())     # inherited method: Whiskers makes a sound
print(whiskers.purr())      # Whiskers purrs
```

3.6 Method Overriding

A subclass can provide its own implementation of a method already defined in its parent class, replacing (overriding) the inherited behaviour. The subclass can still call the parent's version with `super()` if needed.

```
class Animal:
    def speak(self):
        return "Some generic animal sound"

class Dog(Animal):
    def speak(self):                        # overrides Animal.speak
        base = super().speak()
        return f"{base}, specifically: Woof!"

print(Dog().speak())
```

3.7 Polymorphism

Polymorphism means that objects of different classes can be treated through a common interface, with each class responding to the same method call in its own way. This is powerful because calling code does not need to know the exact type of an object, only that it supports a certain method.

```
class Cat:
    def speak(self):
        return "Meow"

class Dog:
```

```
    def speak(self):
        return "Woof"

animals = [Cat(), Dog(), Cat()]
for animal in animals:
    print(animal.speak())     # each object 'speaks' differently
```

4. Syntax Summary

```
class ClassName:
    def __init__(self, param1, param2):
        self.attribute1 = param1
        self.attribute2 = param2

    def instance_method(self):
        return self.attribute1

class SubClass(ClassName):
    def instance_method(self):        # overriding
        return super().instance_method() + " extended"

obj = ClassName(value1, value2)
obj.instance_method()
```

5. Code Examples

Example 1: Simple Rectangle Class

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

rect = Rectangle(5, 3)
print(rect.area(), rect.perimeter())
```

Example 2: Employee Inheritance Hierarchy

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def annual_salary(self):
        return self.salary * 12

class Manager(Employee):
    def __init__(self, name, salary, bonus):
```

```
        super().__init__(name, salary)
        self.bonus = bonus

    def annual_salary(self):
        return super().annual_salary() + self.bonus

m = Manager("Priya", 8000, 5000)
print(m.annual_salary())
```

Example 3: Polymorphic Shape Areas

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

class Square:
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

shapes = [Circle(3), Square(4)]
for shape in shapes:
    print(round(shape.area(), 2))
```

6. Step-by-Step Explanation

Walking through Example 2 (Employee Inheritance) line by line:

1. `Manager(Employee)` declares that Manager inherits everything defined in Employee.
2. Inside Manager's `__init__`, `super().__init__(name, salary)` calls the parent class's constructor to set up the inherited attributes, avoiding duplicated code.
3. `self.bonus = bonus` adds a new attribute specific to Manager.
4. Manager overrides `annual_salary()`, but reuses the parent's calculation via `super().annual_salary()` and adds the bonus on top.
5. When `m.annual_salary()` is called, Python uses Manager's overridden version, not Employee's original one.

7. Common Mistakes

■ **Common Mistake:** forgetting to include self as the first parameter of instance methods, causing a TypeError when called.

■ **Common Mistake:** forgetting to call super().__init__() in a subclass constructor, leaving inherited attributes unset.

■ **Common Mistake:** confusing class attributes (shared by all instances) with instance attributes (unique per object), leading to unexpected shared state.

■ **Common Mistake:** assuming double-underscore attributes are truly private — Python only name-mangles them; they remain accessible via _ClassName__attribute.

■ **Common Mistake:** overriding a method without matching the expected signature, breaking code that relies on the parent class's interface.

8. Best Practices

✔ **Best Practice:** keep classes focused on a single responsibility, following the same principle used for functions.

✔ **Best Practice:** use inheritance for genuine 'is-a' relationships; prefer composition ('has-a') when that fits the problem better.

✔ **Best Practice:** always call super().__init__() in a subclass to ensure the parent class is properly initialized.

✔ **Best Practice:** expose behaviour through methods rather than letting external code manipulate internal attributes directly.

9. Practice Questions

1. What is the purpose of the __init__ method, and when is it automatically called?
2. Why does every instance method need self as its first parameter?
3. What is the difference between a class attribute and an instance attribute?
4. How does super() help avoid code duplication in a subclass constructor?
5. Give an example of polymorphism that does not involve inheritance.

10. Mini Coding Exercises

1. Create a class Book with title, author, and pages attributes, and a method summary() that returns a formatted description.
2. Create a class Vehicle with a method describe(), then create subclasses Car and Motorcycle that override describe().
3. Create a class Counter with a private attribute __count, and methods increment() and get_count().
4. Create a base class Shape with an area() method that raises NotImplementedError, then implement Circle and Rectangle subclasses.
5. Write a loop that calls a shared method on a list of different objects (polymorphism) and prints each result.

11. Interview / QC Questions

1. What is the difference between encapsulation and abstraction in object-oriented design?

2 Explain the difference between method overriding and method overloading, and note whether Python supports true overloading.

3 What does the super() function do, and why is it preferred over calling the parent class directly by name?

4 Describe a scenario where composition would be a better design choice than inheritance.

5 What is duck typing, and how does it relate to polymorphism in Python?

12. Summary

This module introduced object-oriented programming in Python: defining classes as blueprints for objects, using __init__ and self to initialize and access instance-specific data, and writing instance methods that operate on that data. You learned how encapsulation protects internal state through naming conventions, how inheritance lets subclasses reuse and extend a parent class's behaviour, how method overriding customizes inherited methods, and how polymorphism allows different object types to be used interchangeably through a shared interface. These concepts let you model complex systems in a structured, extensible way and are foundational to most real-world Python applications and frameworks.

End of Python_OOP — continue to Python_File_Handling.pdf for the next module.